6

CURSO PARA INTEGRADORES DO HUBSAÚDE · MÓDULO 6 DE 6

Projeto final

Nos módulos anteriores, você executou cada etapa da integração separadamente. No projeto final, você leva esse conteúdo para o seu próprio sistema: ele deve montar, conferir, autenticar, enviar e acompanhar um registro no Simulador, de forma automática e com tratamento das falhas mais comuns. Ao final, o sistema estará pronto para receber a credencial oficial.

AO FINAL, VOCÊ TERÁ

- ✓ um componente do seu sistema que realiza o fluxo completo de envio ao Simulador;
- ✓ o tratamento das recusas, das falhas de rede e da expiração do token;
- ✓ as evidências, colhidas no Simulador, de que cada parte funciona;
- ✓ a lista do que muda quando o credenciamento for aprovado.

COMO O MÓDULO ESTÁ ORGANIZADO

1. **O desafio:** enunciado e ponto de partida
2. **Comprovação:** seis critérios de conclusão
3. **Passagem para a produção:** o que entregar, confirmar e trocar
4. **Verificação:** perguntas, glossário e respostas

VOCÊ VAI PRECISAR

- O ambiente dos módulos 2 a 5: o Simulador, o sistema cadastrado e os certificados de teste.
- O programa `EnviarPaciente.java`, do módulo 5, como ponto de partida.
- O seu sistema, ou uma cópia dele, e apenas dados fictícios.
- Este módulo não tem vídeo: é um desafio para ser resolvido por você.

DÚVIDAS E SUGESTÕES

Se algum passo ficar difícil, escreva para o suporte do HubSaúde, em `suporte@saude.go.gov.br`, com o assunto "Curso para integradores, projeto final". A versão mais recente deste módulo, com os vídeos, está em `https://sesgo-ti.github.io/hubsaude/curso/projeto-final/`.

Copiar de um PDF pode alterar aspas, hífens e quebras de linha. Para os comandos e arquivos, prefira a versão do módulo no portal ou os arquivos para baixar, em `https://sesgo-ti.github.io/hubsaude/curso/materiais/`.

PARTE 1

O desafio

6.1 Como funciona o desafio

O projeto final não traz um passo a passo. Ele traz um enunciado, com os requisitos que o componente deve cumprir, e um conjunto de critérios de conclusão, cada um com uma evidência objetiva que você confere no Simulador.

1. **Implementar**
   Desenvolva o componente no seu sistema, seguindo o enunciado da seção 6.2. Os arquivos e comandos dos módulos anteriores servem de ponto de partida.
2. **Comprovar**
   Para cada critério de conclusão, execute o componente contra o Simulador e confira a evidência indicada: o estado da Task, a resposta ou o log.
3. **Corrigir**
   Se algo não sair como esperado, procure a mensagem na tabela de erros mais comuns. Cada uma traz a causa e a correção.

6.2 O enunciado

Escolha um tipo de informação que o seu sistema produz, como o cadastro de pacientes, e desenvolva, na linguagem do sistema, um componente que cumpra os oito requisitos abaixo. Use apenas dados fictícios.

O fluxo principal

Os cinco primeiros requisitos fazem o registro chegar ao HubSaúde:

Requisito	O que o componente faz	Visto no
1. Montar	Converte os dados do próprio sistema em um registro FHIR.	Módulo 3
2. Conferir	Valida o registro com `hubsaude validate -o json` antes do envio e interrompe o processo quando houver erros.	Módulo 3
3. Autenticar	Obtém o token com a biblioteca oficial, reutilizando o mesmo cliente durante toda a execução.	Módulo 4
4. Enviar	Envia com os cabeçalhos obrigatórios e uma `Idempotency-Key` gerada uma única vez e guardada até a confirmação.	Módulo 5
5. Acompanhar	Consulta a Task, respeitando o intervalo sugerido em `Retry-After`, até o resultado.	Módulo 5

A robustez

Os três últimos requisitos distinguem um componente de demonstração de um componente pronto para a produção:

Requisito	O que o componente faz	Visto no
6. Recusas	Registra em log o motivo informado pelo HubSaúde.	Módulo 5
7. Expiração do token	Diante de um 401, descarta o token guardado e tenta novamente uma única vez.	Módulo 4
8. Falhas de rede	Repete o envio com a mesma `Idempotency-Key`.	Módulo 5

6.3 Do exercício ao componente

O programa `EnviarPaciente.java`, do módulo 5, já percorre o fluxo principal, mas foi escrito para o exercício. Compare o que ele faz com o que o enunciado pede:

Requisito	O programa do módulo 5	O que falta no componente
1. Montar	Lê um arquivo JSON pronto.	Montar o registro a partir dos dados do sistema.
2. Conferir	Não valida.	Executar o Validador antes do envio e decidir pelo código de saída: 1 interrompe.
3. Autenticar	Cria o cliente uma vez.	Manter um único cliente durante toda a execução, e não um por envio.
4. Enviar	Gera uma chave nova a cada execução.	Gerar a chave uma vez por registro e guardá-la com ele até a confirmação.
5. Acompanhar	Espera sempre dois segundos.	Respeitar o `Retry-After` da resposta.
6. Recusas	Exibe o motivo na tela.	Registrar o motivo no log do sistema.
7. Expiração	Não trata o 401.	Descartar o token e tentar de novo uma única vez.
8. Rede	Interrompe ao primeiro erro.	Repetir com a mesma chave; diante de 409, aguardar.

Uma organização sugerida

Separar o componente em quatro partes facilita os testes, porque cada parte pode ser conferida isoladamente com os arquivos dos módulos anteriores:

Montagem

Converte os dados do sistema em FHIR. Testada com o Validador, sem nenhum envio.

Conferência

Chama o Validador e devolve se o registro pode seguir. Testada com o `paciente-com-erros.json`.

Cliente

Obtém o token e faz as chamadas HTTP com o certificado de cliente.

Acompanhamento

Consulta a Task e interpreta o resultado: aceito, recusado ou ainda em processamento.

DICA

Mantenha a configuração fora do código, em variáveis de ambiente ou em um arquivo de configuração: o endereço do servidor, o `client_id`, a chave, o certificado e o certificado de confiança. É isso que permitirá passar à produção sem alterar o código, como mostra a última parte deste módulo.

PARTE 2

Comprovação

Mantenha um terminal aberto com o log do Simulador durante os testes. Cada envio aceito, cada repetição e cada recusa aparecem nele:

VOCÊ DIGITA

```
$ hubsaude sim logs -f
```

A Task de um envio é consultada pelo endereço completo devolvido em `Content-Location`. O Simulador não oferece busca de Task: uma consulta sem o identificador é recusada. Marque cada critério quando a evidência aparecer.

1 Registro aceito

O componente envia um registro válido e chega ao registro criado.

Execute o componente com dados fictícios válidos.

Onde conferir	O que deve aparecer
A Task	`status` igual a `completed`; `owner` apontando para o seu sistema, `Organization/meu-sistema`; `output` com a referência do registro criado, como `Patient/` seguido do identificador.
O log do Simulador	Uma linha `POST Patient aceito`, com o identificador da Task e a `Idempotency-Key` usada.

☐ **Passo concluído**

2 Conferência antes do envio

Um registro com erro é barrado antes de chegar ao Simulador.

Altere os dados para produzir um registro inválido, como o sexo informado com o código `F` no lugar de `female`.

Onde conferir	O que deve aparecer
O componente	O processo é interrompido após a validação, com o problema apontado.
O log do Simulador	Nenhuma linha nova de `POST Patient aceito`: o registro não foi enviado.

☐ **Passo concluído**

3 Recusa tratada

Uma recusa do Simulador é registrada com o motivo.

Desative temporariamente a conferência e envie o mesmo registro inválido.

Onde conferir	O que deve aparecer
A resposta do envio	202: o registro foi aceito para processamento.
A Task	`status` igual a `rejected`, `statusReason` "Falha na validação HAPI FHIR" e, no OperationOutcome contido, `HAPI-1821: [element="gender"] Invalid attribute value "F"`.
O log do componente	O motivo da recusa.

> **ATENÇÃO**
> Volte a ativar a conferência depois deste teste.

☐ Passo concluído

4 Repetição sem duplicidade

Um envio repetido com a mesma chave não cria um segundo registro.

Repita um envio já concluído com a mesma `Idempotency-Key`, por exemplo interrompendo o componente logo após o envio e executando-o novamente.

Onde conferir	O que deve aparecer
A resposta	202, com o mesmo endereço de Task e o cabeçalho `Idempotency-Replayed: true`.
O log do Simulador	`Replay pos-conclusao detectado`.

> **OBSERVE**
> Se a repetição chegar enquanto o envio original ainda está em processamento, a resposta é 409 e o log mostra `Replay concorrente detectado`. O componente deve aguardar e tentar novamente.

☐ Passo concluído

5 Expiração do token tratada

O componente continua funcionando depois que o token expira.

Mantenha o componente em execução por mais de cinco minutos, enviando registros periodicamente.

Onde conferir	O que deve aparecer
O log do componente	Envios aceitos depois dos cinco minutos, sem nenhuma resposta 401: a biblioteca renova o token automaticamente.

☐ Passo concluído

6 Configuração externa

Endereço, client_id e certificados podem ser trocados sem alterar o código.

Troque um dos valores da configuração, como o `client_id`, por um valor inexistente, e execute o componente.

Onde conferir	O que deve aparecer
O componente	A obtenção do token falha com `invalid_client`, sem nenhuma alteração no código. Com o valor correto de volta, o componente volta a funcionar.

> **OBSERVE**
> Essa é a condição para passar à produção sem alterar o código.

☐ **Passo concluído**

6.4 Erros mais comuns

Cada linha traz a mensagem como ela aparece na resposta, a causa e a correção.

Mensagem	Causa	Correção
400: `Header 'Idempotency-Key' é obrigatório`	O envio foi feito sem a chave.	Gerar uma chave única, como um UUID, antes do primeiro envio de cada registro, guardá-la e enviá-la em todas as tentativas desse registro.
401: `invalid_token: Token inválido ou expirado`	O token expirou, ou a chamada foi feita sem o certificado de cliente.	Usar o certificado em todas as chamadas. Se o token expirou, obter outro e tentar uma única vez; a biblioteca faz essa renovação.
403: `insufficient_scope: Escopos necessários`	O token não tem o escopo da operação, como `system/Patient.c` para criar.	Pedir o escopo na obtenção do token e confirmar que ele está liberado no cadastro.
`invalid_client` na obtenção do token	`client_id` ou certificado diferentes do cadastro, ou um `client_assertion` reaproveitado.	Conferir o cadastro e gerar um novo `client_assertion` a cada pedido; a biblioteca gera um `jti` novo.
`invalid_scope` na obtenção do token	Um escopo pedido não está liberado para o `client_id`.	Pedir apenas os escopos liberados, escritos exatamente como no cadastro.
Task com `status rejected`	O registro foi recebido, mas recusado no processamento.	Corrigir os dados apontados e enviar com uma **nova** chave, pois é um novo registro. Validar antes evita a recusa.
409 ao repetir um envio	A repetição chegou com o original ainda em processamento.	Aguardar o intervalo sugerido e tentar de novo com a **mesma** chave.

DICA

Note a diferença entre as duas últimas linhas: depois de uma recusa, o registro corrigido é um registro novo e recebe uma chave nova; diante de um 409, é o mesmo envio e a chave é a mesma.

PARTE 3

Passagem para a produção

O credenciamento é conduzido pelo gestor da instituição, conforme a jornada do gestor no portal. Na passagem para a produção, o integrador cuida de três pontos:

O que entregar ao gestor

A parte pública do certificado ICP-Brasil da instituição, em arquivo `.cer`, se você for o responsável por ela. A chave privada nunca é entregue.

O que confirmar

Que a credencial oficial funciona, com o comando `hubsaude auth verify`, antes de alterar o sistema.

O que trocar

Apenas os valores da configuração: endereço, `client_id`, chave e certificado.

A página "Preparar o certificado digital", no portal, explica como gerar e conferir o arquivo `.cer`.

Ensaiar a confirmação no Simulador

O comando `hubsaude auth verify` pode ser ensaiado agora, contra o Simulador, com o certificado de teste. A opção `--trust` faz o comando confiar no certificado do Simulador:

VOCÊ DIGITA

```
$ hubsaude auth verify --fhir-base https://localhost:8443 --client-id meu-sistema \
    --key client-key.pem --cert client-cert.pem --trust simulador-server.pem \
    --ig hemograma --ig-versao 0.0.1 \
    --scope "system/Patient.c system/Patient.rs system/Task.rs"
```

O comando exibe os dados do token obtido e termina com a mensagem:

VOCÊ VÊ

```
[OK] Credenciamento verificado. Acesso ao HubSaúde está funcionando.
```

Na produção, o comando é o mesmo, com o endereço oficial, o `client_id` e o certificado da instituição, e sem a opção `--trust`.

O que muda

	No Simulador	Na produção
Endereço	`https://localhost:8443`	`https://hub.saude.go.gov.br`
`client_id`	`meu-sistema`	O identificador entregue no credenciamento
Chave e certificado	Os de teste	Os da instituição, ICP-Brasil
Confiança no servidor	`simulador-server.pem`	Nenhuma configuração: o certificado do HubSaúde é reconhecido automaticamente
Escopos	Os liberados no cadastro do Simulador	Os autorizados no credenciamento

> **OBSERVE**
> No ambiente oficial, os registros devem seguir os perfis do Guia de Implementação do tipo de informação enviado, e os envios são feitos em um Bundle com o perfil indicado no guia.

PARTE 4

Verificação

Responda sem consultar o texto. As respostas estão no final do módulo.

1. Por que a Idempotency-Key deve ser guardada até a confirmação do resultado, e não gerada a cada tentativa?

2. Um envio recebeu 202, mas a Task terminou como rejected. O registro foi criado?

3. O que precisa mudar no código do componente para passar à produção?

4. Um registro recusado foi corrigido. Ele deve ser reenviado com a mesma Idempotency-Key?

PRÓXIMO MÓDULO

Extra: assinar com o Safira

Opcional. Se o seu sistema também precisa assinar registros, o módulo extra mostra como assinar e conferir assinaturas digitais com o Safira.

Respostas da verificação

1. *Por que a Idempotency-Key deve ser guardada até a confirmação do resultado, e não gerada a cada tentativa?*

Porque é ela que permite ao HubSaúde reconhecer uma repetição. Uma chave nova a cada tentativa faria cada repetição ser tratada como um novo registro, com risco de duplicidade.

2. *Um envio recebeu 202, mas a Task terminou como rejected. O registro foi criado?*

Não. O 202 confirma apenas o recebimento para processamento. O resultado é o da Task: com `rejected`, nada foi criado, e o motivo está em `statusReason` e no OperationOutcome contido nela.

3. *O que precisa mudar no código do componente para passar à produção?*

Nada, se a configuração for externa. Mudam apenas os valores: endereço, `client_id`, chave e certificado. O certificado de confiança do Simulador deixa de ser informado.

4. *Um registro recusado foi corrigido. Ele deve ser reenviado com a mesma Idempotency-Key?*

Não. O registro corrigido é um registro novo e recebe uma chave nova. A mesma chave serve apenas para repetir um envio cujo resultado não se conhece.